

OOPJ Day 9

Thu, 17 Sept 26

Object Class: Key Methods

- Every Java class implicitly inherits from `java.lang.Object`
- Methods available to all objects (all overridable by subclasses):
 - `toString`: returns a string representation of the object
 - `equals`: compares two objects for logical equality
 - `hashCode`: generates a hash value for the object (used in hashing technology)
 - `clone`: creates a copy of the object (deep copy)
 - `finalize`: called before garbage collection; deprecated, do not rely on it
 - `getClass`: used in reflection to get runtime class information
- Thread/synchronization-specific methods: `wait`, `notify`, `notifyAll`
 - These are marked `final` and cannot be overridden by subclasses
- All non-final methods can be customized in any derived class as needed

Memory Management and the JVM

- Normal flow without memory pool:
 - App requests memory from JVM, JVM requests from OS, OS allocates from RAM
 - This round-trip degrades application performance significantly
- JVM solution: pre-allocate a large block (e.g., 5,000 bytes) called the memory pool at startup
 - All object allocations served from this pool directly, no OS round-trip needed
- When the pool is exhausted, JVM has two options:
 - Reclaim unused memory via garbage collection
 - Extend the pool by allocating a larger block from the OS
- Pool extension uses an algorithm based on consumption history:
 - Example: if 5,000 bytes consumed in 10 minutes, next allocation might be 10,000 bytes
- All objects in Java are allocated on a managed heap, which is the memory pool managed by the JVM
- Java provides no delete operator and no destructor; the programmer does not control object destruction
 - This is why destructors (as in C++) do not exist in Java

Reference Counting Mechanism

- Every object on the managed heap maintains a reference count (RC)
- RC incremented in three situations:

1. A new object is created and assigned to a reference: `Data obj = new Data()`
 2. An existing reference is assigned to another reference or object: `obj2 = obj`, or `obj = someOtherRef`
 3. An object's address is passed into or returned from a method and stored in a reference
- RC decremented in three situations:
 1. Reference is assigned to null: `obj = null` (link to object is broken)
 2. Reference is reassigned to a different object: `obj = new Data()` (old link broken, RC of old object decremented)
 3. Reference goes out of scope, e.g., a local reference inside an if/while/do block is destroyed when the block ends
 - Both the memory manager and the garbage collector use the reference counting mechanism
 - This entire mechanism is possible because of the managed heap provided by the JVM

Garbage Collection

- GC is a low-priority background thread running inside the JVM
- Continuously monitors the RC of every object on the managed heap
- When RC of an object reaches zero, GC destroys the object and reclaims its memory back to the pool
- Java does not provide a delete operator; the GC is the sole mechanism for object destruction
- `System.gc()`:
 - Does NOT directly invoke the garbage collector
 - Only makes a request to increase the priority of the GC thread
 - JVM decides whether to honor the request based on available memory in the pool
 - If enough memory is available, JVM may ignore the request entirely
- Key factors JVM uses to decide whether to run GC:
 - How much memory is currently available in the pool
 - How much memory can realistically be reclaimed by running GC
 - Example: if 2,000 bytes needed but GC can only reclaim 200 bytes, JVM will extend the pool instead
- GC types: Java provides multiple garbage collector implementations
 - G1 (Generation 1) collector is the current best-performing default
 - GC parameters (frequency, memory threshold, etc.) can be tuned by performance specialists via JVM flags
 - Most developers do not need to configure this
- Finalization: older GC versions invoked `finalize()` on objects before collection
 - This process is deprecated; do not rely on `finalize()` for cleanup logic

Exception Handling: Concepts and Keywords

- An exception is an unpredictable runtime situation that causes abnormal or abrupt program execution
 - Examples: user enters invalid input, a null reference is dereferenced, array index exceeds bounds
 - Real-world analogy: tire going flat while driving; the flat tire is the exception, changing to the spare is the handling
- Exception handling is a module inside the JVM that monitors exceptions and controls application flow when they arise
- Unlike C++, Java cannot throw or catch primitive data types; every exception must be an object that is directly or indirectly inherited from Throwable

Five Keywords

1. try:
 - A block of code that you believe has a possibility of raising an exception
 - When an exception is raised inside try, remaining statements in the block are skipped
 - Control automatically flows to the appropriate catch block
 - Example: opening a file, reading from it, writing to a database, all inside one try block
2. catch:
 - A block of code responsible for handling a specific exception type thrown by the try block
 - A single try block can have multiple catch blocks, one per exception type
 - The programmer defines the counteraction logic inside catch
 - Example: showing a dialog box asking “Do you want to overwrite this file?” when a duplicate filename is detected
 - A catch block can handle multiple unrelated exception types using | (introduced in Java 7):
 - `catch (IOException | SQLException e)`
 - When handling exceptions that share an inheritance relationship, always handle the most-derived class first, then the base class
 - If the base class catch appears first, it will catch everything and the derived catch becomes unreachable (compilation error)
3. throw:
 - A keyword used to explicitly raise an exception from a specific statement
 - Typically used inside a try block or a method when an invalid condition is detected
 - Example: `throw new DateException("Invalid value of day")`
 - In Java, you throw an object, not a primitive
4. throws:
 - A keyword used in the declaration of a method
 - States that this method has the capability of raising one or more exceptions
 - Delegates the responsibility of handling the exception to the caller of the method

- Example: `public void sendDate(int day, int month, int year) throws DateException`
 - If a method throws a checked exception, the caller must handle it (try-catch) or further declare throws
 - It does not make sense for a method to both throw and catch its own exception internally
5. finally:
- A block of code that executes irrespective of whether an exception was raised or not
 - Guaranteed to execute before control leaves the try-catch block
 - Solves the problem of duplicating cleanup code (e.g., closing a file) in both the try and catch blocks
 - Example: file open in try, exception on write, catch handles error, finally closes the file regardless
 - Important: if an exception is uncaught and propagates out of the try-catch, finally still executes before the exception moves up

Exception Hierarchy

- Throwable is the base class for all exceptions in Java
- Throwable branches into two types:

Error

- Represents severe runtime problems
- Programmer can catch an Error but typically cannot do anything meaningful to recover
- Examples:
 - `OutOfMemoryError`: JVM cannot allocate requested memory, OS RAM is full
 - `BadVersionError`: bytecode version incompatible with JVM
 - `UnresolvedCompilationError`
- These are situations outside the programmer's control

Exception

- Represents conditions the programmer can handle and respond to meaningfully
- Logically divided into two categories:

Checked Exceptions

- The compiler forces you to handle these; failure to do so causes a compilation error
- All classes that directly extend `Exception` (not `RuntimeException`) are checked
- Examples: `IOException`, `SQLException`, `SocketException`, `FileNotFoundException`
- Example error: "Unhandled exception type `IOException`" when calling `System.in.read()` without a try-catch

- The method declaration uses throws to signal to callers that a checked exception may be raised

Unchecked Exceptions

- The compiler does NOT force you to handle these
- All classes that directly or indirectly extend RuntimeException are unchecked
- Examples: NullPointerException, ClassCastException, ArrayIndexOutOfBoundsException
- Example: Data obj = null; obj.toString() compiles fine but throws NullPointerException at runtime
- No compilation error even without a try-catch block

Why So Many Exception Classes (~2,500+ in Java)?

- Most exception classes contain only constructors and no additional logic
- The purpose of each class is to identify the nature of the problem, not to add behavior
- Analogy: tire flat, petrol over, engine failure are three different problems requiring three different responses
 - You can only respond correctly if you can identify which problem occurred
- RTTI (Runtime Type Identification) is used internally to match the thrown object to the correct catch block
- Always handle exceptions from the most-derived class to the most-base class in a sequence of catch blocks

Control Flow, Nesting, and Exception Chaining

Basic Control Flow

- Normal execution (no exception): try executes fully, catch is skipped, finally executes, program continues
- Exception raised: remaining try statements skipped, control jumps to matching catch, finally executes, program continues
- If no matching catch block exists: exception is uncaught and propagates to the caller of the current method
 - If uncaught all the way to main, JVM prints the exception and terminates the program
 - Statement after the try-catch block is NOT executed if the exception escapes

Multiple Catch Blocks

- A single try block can raise different types of exceptions at different points
- Each exception type gets its own catch block

- Example: opening a file can raise `IOException`, opening a DB connection can raise `SQLException`
- Execution: the moment one exception is raised, control jumps to the matching catch; remaining try statements are skipped
- Java 7+ multi-catch: `catch (IOException | SQLException e)` handles two unrelated types in one block
 - Cannot use this syntax for exception types that share an inheritance relationship

Nested Try-Catch

- Use nested try-catch when inner statements are interdependent and you want to isolate failure scope
- Practical use case (menu-driven program):
 - Outer try-catch wraps the main menu loop
 - Inner try-catch wraps the logic for each menu option (e.g., add employee)
 - If an exception occurs in the inner block, only the inner loop breaks; the outer menu loop continues
 - Without inner try-catch, any exception would propagate to the outer catch and terminate the entire menu
- Rule: statements that depend on each other should be grouped in their own inner try-catch

Exception Chaining (Rethrowing)

- After handling an exception in a catch block, you can rethrow it to the caller for further handling
- Two approaches:
 1. Rethrow as-is: `throw e` (passes the original exception object up the call chain)
 2. Rethrow with added context: `throw new SomeException("additional message", e)`
 - Wraps the original exception inside a new one with extra information
 - Analogy: teacher handles a student's misbehavior, then reports it to a senior with additional remarks; senior escalates further
- When rethrowing, the finally block of the current try-catch still executes before the exception propagates
- Any statements between the rethrow and the end of the catch block are skipped

User-Defined Exceptions

- You can create custom exception classes to represent domain-specific error conditions
- Steps:
 - Create a new class
 - Decide: checked or unchecked?
 - Extend `Exception` for a checked exception (compiler forces callers to handle it)

- Extend RuntimeException for an unchecked exception (compiler does not force handling)
 - Add constructors as needed (e.g., one that takes a message, one that takes a message and a cause)
 - Most logic lives in the parent class; the custom class mainly provides identity and constructors
- Example:
 - class DateException extends Exception with a constructor taking a String message
 - In the Date class: if (day < 1 || day > 31) throw new DateException("Invalid value of day")
 - Method declaration: public void setDate(int day, int month, int year) throws DateException
 - Caller must wrap the setDate call in a try-catch block
- Switching to RuntimeException as the parent makes it unchecked: no compilation error if caller does not handle it, but exception still thrown at runtime

Upcoming Topics and Lab Info

- Next session topics:
 - File operations (persistence and serialization)
 - Collections framework: what it is, why it exists, how to use it
- Lab session starts at 2:00 PM today (individual labs, not common)
- Interviews at 2:00-2:30 PM for: Pang, Yash, Shreya, and Sweetie
- Pending evaluations will be called separately
- Code and whiteboard from today's session shared via chat by Pankaj sir
- Handwritten notes recommended for interview and placement preparation

Chat with meeting transcript:

<https://notes.granola.ai/t/684b5ca4-7ab9-4445-bf65-f4c5674adbe7-00demib2>